

MEETLY AI

An AI meeting notetaker

Record a conversation and get back a transcript that plays in sync, an AI summary you can re-cut by template, action items, highlights you can mark while the call is still running, search across every meeting, and a public clip link that works for someone who was never invited. Built as a 24-hour rebuild of fathom.video, then redesigned as its own product rather than a copy of one.

23	9	234	8.3k
API ENDPOINTS	DATABASE TABLES	TRANSCRIPT SEGMENTS	LINES OF CODE

The pipeline is real, and so is the backend.

Meetly captures the meeting tab in your browser - every participant, not just your microphone - stores the audio, transcribes it with Whisper, summarises it, and cuts real clips with ffmpeg. Every endpoint reads and writes a live database; nothing returns a canned payload. Where something is a placeholder, the product says so on screen and this document names it.

Muhammad Umar · React, TypeScript, Tailwind · FastAPI, SQLAlchemy, SQLite

Contents

What this is	3
What it does	3
What I left out, deliberately	3
Design decisions	4
An identity, not a theme	4
The meeting page	4
Lists are ledgers	4
Search is two panes	4
Architecture	5
Stack	5
Data model	5
The processing pipeline	5
What is real and what is not	6
How to check the backend yourself	6
Engineering notes	7
How it was built	8
Agent capture	8
Running it	8
Deployment	8
If I had another day	8

What this is

The brief was to rebuild fathom.video in 24 hours, use the real product first, and spend the time on whatever mattered most. Faking the capture layer was explicitly allowed. A later note from the reviewers changed the target: keep the idea and the backend, but make the design my own rather than a clone.

So this is not a copy of Fathom's interface. The information architecture, the visual language and the page layouts are decisions I made and can defend. The product problem is the same one Fathom solves: a meeting happens, and afterwards nobody can find the part that mattered.

What it does

Capability	How it works
Recording	Browser capture of the shared meeting tab mixed with your microphone, so remote participants are recorded, not just the room you sit in.
Mid-call marking	A button during the recording. Each press is stored as elapsed seconds and becomes a highlight once the transcript resolves it.
Transcription	OpenAI Whisper with per-segment timestamps, which is what lets the transcript follow playback and lets any line seek.
Summary	An LLM summary with five templates. Switching template re-reads the transcript rather than re-labelling the same text.
Action items	Extracted with the summary, editable, and countable across meetings.
Highlights	Star any line, or mark it live. Categorised, and playable.
Search	One query across transcripts, titles, summaries, action items, highlights and participants. Results land on the exact second.
Clips	ffmpeg cuts the selected range out of the stored audio into its own file, behind a share link that needs no account.

What I left out, deliberately

- **A bot that joins the call.** Reliably joining Zoom, Meet and Teams is days of infrastructure work and no product insight. Sharing the meeting tab records the same audio. The brief permits the stub; I took it and said so.
- **Accounts and login.** The brief requires the live link to open for someone who is not signed in. A login wall works directly against that. Adding auth later means a `user_id` column and a filter on each query, not a rewrite.
- **Semantic search.** Keyword search across six sources answers the real question - *where was that said* - and ships in hours, not days.

Design decisions

The second brief asked for my own design choices, so every one below is a choice, not a default.

An identity, not a theme

Most meeting tools are blue-on-white and read like dashboards. Meetly is warm neutral paper with a burnt-amber accent. That palette does one specific job: a meeting archive is something you read, and reading surfaces should not glare. The accent is reserved for the two things that are genuinely actionable - the playhead and the primary action - so it never becomes decoration.

Surfaces are square. Rounded cards make everything look like a widget; hairline rules and flat panels make a long list look like a document, which is what it is.

The meeting page

Two columns. The player and its tabs sit on the left; the transcript is a rail on the right that stays visible while you move through the summary, the action items and the highlights. The transcript is the source of truth for everything else on the page, so it should not be a tab you leave.

Playback and transcript are bound in both directions. Playing scrolls the transcript; clicking a line seeks. Scrolling away quietly turns following off, and jumping from a highlight or the summary turns it back on, because a jump is a deliberate request to go somewhere.

Lists are ledgers

Meetings, the dashboard and the calendar all use the same row: a fixed left column with the clock time, then the title and one line of summary. Because the column is fixed, every row in a long list lines up on the same axis and the page can be scanned without reading it. Rows are grouped under day headings that stick to the top while you scroll, so you always know which day you are looking at.

Filters on the meetings page live in a rail rather than a toolbar, because they are the state of the page, and a rail keeps both the options and the current value visible while you scroll. Each person is listed with a count taken from the unfiltered set, so a name never reads as zero just because another filter is hiding it.

Search is two panes

Which meetings a phrase turns up in is itself an answer, so the breakdown is a rail of meetings with hit counts that also filters the results, rather than a line of prose above a flat list.

A design decision I reversed

I first built the meeting page around a speaker-lane timeline: one horizontal lane per person, showing who dominated and where the room went quiet. It was the most interesting thing in the build and I was attached to it. Shown the running product, it read as a data visualisation sitting where a player belonged - you could see the shape of the call but not watch it. I put the recognisable player back. The timeline is still the better artefact; the player is the better product.

Architecture

Stack

Layer	Choice	Why
Frontend	React 19, TypeScript, Vite, Tailwind v4	Tailwind tokens defined once in @theme, so the whole identity is one file.
Server state	TanStack Query	Cache invalidation and polling for free. A processing meeting polls until ready and stops on its own.
API	FastAPI, Pydantic v2	Response models are the schema and the validation and the docs.
Database	SQLAlchemy over SQLite	Runs with no setup. Nothing depends on SQLite specifically - the same models work against Postgres.
Media	Files on disk, served with HTTP Range	Range support is what makes seeking work at all; without 206 responses a browser cannot scrub.
Audio cutting	ffmpeg via the imageio-ffmpeg wheel	No system install. Clips re-encode rather than stream-copy, so a cut lands on the requested second instead of the nearest keyframe.

Data model

Nine tables. A meeting owns its participants, transcript segments, summary, action items and highlights; clips reference a meeting and carry their own cut file; settings hold the runtime AI configuration.

```
meetings → participants, transcript_segments, summaries, action_items, highlights, clips | settings, users
```

Schema changes run through an idempotent ALTER TABLE check on boot rather than a migration tool. For a project this size Alembic is ceremony; the check adds a missing column and does nothing when it is already there.

The processing pipeline

Upload returns immediately and the work happens in a background task, because transcription takes seconds to minutes and a request should not hold that open. The meeting is created straight away with a processing status, the UI polls it, and any failure is written onto the meeting so the page can say what broke instead of spinning on "Processing" forever.

```
audio → storage.save() → whisper (segment timestamps) → summarise → resolve mid-call marks → ready
```

What is real and what is not

The reviewers asked for a backend that is genuinely connected - a working database and API, not mock data or hardcoded responses. This section is the honest accounting.

	Status
Database and API	Real. Every endpoint reads and writes SQLite through SQLAlchemy. No endpoint returns a canned payload.
Browser recording	Real. Captures the shared meeting tab - all participants - mixed with your microphone, with a live input meter as proof audio is arriving.
Mid-call marking	Real. Marks travel with the upload and resolve against the transcript once it exists.
Media storage	Real. Audio written to disk under a generated filename and served with Range support.
Transcription	Real with an API key (Whisper, per-segment timestamps). Without a key nothing is transcribed and the transcript says so.
AI summary	Real. LLM with a key; local extraction from the transcript without one. The footer names which path produced what you are reading.
Clips	Real. ffmpeg cuts the range out of the stored audio into its own file.
Public sharing	Real , and genuinely public - no account needed.
Demo meetings	Authored. Eight meetings written by hand so the product demonstrates before you record anything. They are rows in the same database, created by the same seeder, and badged "Demo data".
Calendar OAuth	Placeholder , and the only one. Labelled in the UI and in the API response itself, which carries simulated: true. The events it reports are read from the database.
A bot that joins calls	Not built. Tab sharing records the same audio.
Accounts	None. Deliberate - see the reasoning above.

How to check the backend yourself

Change a row directly in the database and read it back through the API. The second read returns the new value without restarting anything, because the API is reading the row rather than a fixture:

```
sqlite3 backend/meetly.db "update meetings set title='CHANGED' where id=2"
curl -s localhost:8000/api/meetings/2
```

The reverse works too - create a highlight over HTTP and it appears in the table. Search is a real ILIKE across five tables, not a filtered constant.

Seed data is not mock data

The demo meetings are content, not a mocked API. They are inserted by seed.py into the same tables a recording writes to, and they behave identically: you can delete one, star a line in it, cut a clip from it, or find it in search, and every one of those goes through the database. An empty product demonstrates nothing, which is why the brief asks for seeded data; a fake backend demonstrates dishonesty, which is why the follow-up asks against it. Those are different things.

Engineering notes

A few problems that were more interesting than they looked.

Highlights that played the wrong line

Seeded highlights originally carried hand-written timestamps, while segment times are generated from speaking pace. Any wording change shifted the transcript and the timestamps silently drifted, so a highlight would play a line near the one it quoted. Highlights are now anchored to a **quote** and the seeder resolves it to a real time. If a quote stops matching, seeding fails loudly rather than shipping a highlight that plays the wrong moment.

A mark lands on the line being spoken

You press the button a beat *after* the thing worth keeping was said. Snapping to the nearest segment start would therefore often jump forward to the next speaker. A mark resolves to the line that was in progress at that second instead, which is the line you meant.

Following playback, and knowing when to stop

The transcript scrolls itself during playback. Scrolling by hand turns that off - you are reading something else now - and a jump from the timeline, a highlight or the summary turns it back on. The scroll is applied to the list container directly rather than with `scrollIntoView`, which drags every scrollable ancestor and would jerk the whole page on each line.

Deleting a meeting on Windows

A locked media file raised `WinError 32` midway through the delete and aborted the whole transaction, leaving the meeting half-removed. Database rows are now deleted first and files removed afterwards on a best-effort basis, with a sweep on boot that drops media nothing references any more.

A seeding fix that would have deleted your API key

Seeding on first boot dropped and recreated the tables when the meetings table was empty. Once the workspace could legitimately start empty, that meant every restart wiped the settings table too - including a saved API key. Seeding now only fires on a database nobody has touched: no meetings *and* no settings.

Search returning nothing for obvious words

Queries like "deployment" and "deadline" found no hits. The instinct was to loosen matching - stemming, fuzzy distance. The actual cause was that the seeded conversations never used those words. Fixing the corpus was right; loosening the matcher would have hidden the problem and made every other query worse.

How it was built

Agent capture

A capture hook was installed before any code was written, and the test was verified passing first. Every prompt and every final response is written verbatim to `.agent-logs/`, committed as the work happened rather than in one lump at the end. Failures in the hook append to an error log instead of being swallowed, so a silent gap in the record is not possible without evidence of it.

Running it

Backend

```
cd backend
pip install -r requirements.txt
python seed.py --demo
uvicorn app.main:app --reload --port 8000
```

Frontend

```
cd frontend
npm install
npm run dev
```

Then open Settings and paste an OpenAI key to turn on Whisper transcription and AI summaries. The key is stored in the database and read per request, so it applies without a restart. It is never read from the environment - the server actively clears `OPENAI_API_KEY` on boot, so the only key in play is the one you configured.

Deployment

The two halves deploy independently: a `render.yaml` for the API and a `vercel.json` for the frontend, including the rewrite that keeps a share link working on a hard refresh. The API seeds itself on a genuinely fresh database so a new deployment is never an empty workspace.

If I had another day

- **Speaker diarisation.** Whisper gives segments, not speakers. Real per-speaker attribution on recorded meetings is the single biggest gap against the authored ones.
- **Semantic search.** Keyword search finds the phrase you remember; embeddings would find the moment you only remember the shape of.
- **Shared highlight reels.** Clips are single moments. A reel of the five that mattered is what somebody who missed the call actually wants.
- **Real calendar OAuth.** The endpoint and the connection state are already shaped for it; what is missing is a verified provider app, not product thinking.